

Estruturas de Dados em Python

Tuplas, Dicionários, Conjuntos e Matrizes

Material didático sobre sintaxe, propriedades, operações, exemplos e situações adequadas para o uso de cada estrutura.

Por que existem diferentes estruturas de dados?

Cada estrutura organiza os dados de uma maneira diferente. Algumas preservam a ordem, outras relacionam chaves e valores, eliminam repetições ou representam informações distribuídas em linhas e colunas.

TUPLAS

Dados fixos

DICIONÁRIOS

Chave e valor

CONJUNTOS

Valores únicos

MATRIZES

Linhas e colunas

1. Tuplas

O que são?

Uma tupla é uma coleção ordenada de elementos. Ela se parece com uma lista, mas possui uma diferença importante: seus elementos não podem ser alterados após a criação.

Sintaxe

```
nome_da_tupla = (elemento1, elemento2, ...)
```

Quando usar?

Use tuplas quando os dados devem permanecer fixos, como coordenadas, dias da semana, configurações ou valores que não devem ser modificados.

```
# Criando uma tupla
```

```
pokemons = (  
    "Pikachu",  
    "Charmander",  
    "Bulbasaur"  
)
```

```
print(pokemons)  
print(pokemons[0])  
print(len(pokemons))
```

Saída no terminal

```
('Pikachu', 'Charmander', 'Bulbasaur')  
Pikachu
```

Propriedades das tuplas

Ordenadas

Os elementos mantêm uma posição definida.

Imutáveis

Não permitem adicionar, remover ou alterar elementos diretamente.

Permitem repetição

Um mesmo valor pode aparecer mais de uma vez.

Permitem tipos diferentes

Podem armazenar textos, números e valores lógicos.

Acesso por índice

Os índices começam em zero.

Podem ser percorridas

Funcionam com laços for.

Métodos principais

Método	Função
<code>count(x)</code>	Conta quantas vezes um valor aparece.
<code>index(x)</code>	Retorna o índice da primeira ocorrência.

Atenção: uma tupla com apenas um elemento precisa de vírgula: `tupla = ("Pikachu",)`.

2. Dicionários

O que são?

Um dicionário armazena dados em pares formados por uma chave e um valor. A chave funciona como um identificador utilizado para acessar a informação.

```
pokemon = {  
    "nome": "Pikachu",  
    "tipo": "Elétrico",  
    "nivel": 25  
}
```

Sintaxe

```
dicionario = {"chave": valor}
```

```
print(pokemon)  
print(pokemon["nome"])  
print(pokemon["nivel"])
```

Quando usar?

Use dicionários para representar dados estruturados, como cadastros, fichas de estudantes, produtos, personagens ou registros de um sistema.

Saída no terminal

```
{'nome': 'Pikachu', 'tipo': 'Elétrico', 'nivel': 25}  
Pikachu
```

Propriedades dos dicionários

Chave e valor

Cada informação é identificada por uma chave.

Mutáveis

Permitem adicionar, alterar e remover pares.

Chaves únicas

Uma chave não pode aparecer duplicada.

Valores podem repetir

Diferentes chaves podem armazenar o mesmo valor.

Preservam a ordem de inserção

Nas versões atuais do Python.

Chaves devem ser imutáveis

Textos, números e tuplas podem ser usados como chave.

Operações principais

```
pokemon = {  
    "nome": "Charmander",  
    "nivel": 12  
}
```

```
# Adicionar
```

```
pokemon["tipo"] = "Fogo"

# Atualizar
pokemon["nivel"] = 13

# Remover
del pokemon["tipo"]

print(pokemon)
```

Saída no terminal

```
{'nome': 'Charmander', 'nivel': 13}
```

Métodos úteis

Método	Função
keys()	Retorna as chaves.
values()	Retorna os valores.
items()	Retorna os pares chave e valor.

Método	Função
<code>get(chave)</code>	Acessa uma chave sem gerar erro quando ela não existe.
<code>pop(chave)</code>	Remove uma chave e retorna seu valor.
<code>clear()</code>	Remove todos os pares.

3. Conjuntos

O que são?

Um conjunto é uma coleção de elementos únicos. Ele elimina automaticamente valores repetidos e permite realizar operações matemáticas entre coleções.

Sintaxe

```
conjunto = {elemento1, elemento2, ...}
```

Quando usar?

Use conjuntos quando precisar eliminar duplicidades, comparar coleções ou verificar rapidamente se um valor pertence a um grupo.

```
tipos = {  
    "Fogo",  
    "Água",  
    "Elétrico",  
    "Fogo"  
}  
  
print(tipos)  
print(len(tipos))
```

Saída possível no terminal

```
{'Fogo', 'Água', 'Elétrico'}  
3
```

Propriedades dos conjuntos

Não permitem duplicatas

Valores repetidos são eliminados.

Não possuem índice

Não é possível acessar com `conjunto[0]`.

Não garantem ordem visual

A ordem de exibição pode variar.

São mutáveis

Permitem adicionar e remover elementos.

Elementos devem ser imutáveis

Listas e dicionários não podem ser elementos.

Operações matemáticas

União, interseção e diferença.

Operações entre conjuntos

```
equipe_a = {"Pikachu", "Eevee", "Lucario"}  
equipe_b = {"Lucario", "Snorlax", "Gengar"}
```

```
print(equipe_a | equipe_b)  
print(equipe_a & equipe_b)  
print(equipe_a - equipe_b)
```

Saída possível no terminal

```
{'Pikachu', 'Eevee', 'Lucario', 'Snorlax', 'Gengar'}  
{'Lucario'}  
{'Pikachu', 'Eevee'}
```

Operação	Símbolo ou método	Função
União	ou union()	Reúne todos os elementos.
Interseção	& ou intersection()	Retorna os elementos em comum.
Diferença	- ou difference()	Retorna elementos exclusivos do primeiro conjunto.
Diferença simétrica	^	Retorna elementos que não estão nos dois ao mesmo tempo.

Atenção: para criar um conjunto vazio, utilize `set()`. A expressão `{}` cria um dicionário vazio.

4. Matrizes

O que são?

Em Python, uma matriz pode ser representada por uma lista que contém outras listas. Cada lista interna representa uma linha, e cada elemento representa uma coluna.

```
matriz = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]
```

```
print(matriz)  
print(matriz[1][2])
```

Sintaxe

```
matriz = [[...], [...], [...]]
```

Quando usar?

Use matrizes para representar tabelas, mapas, tabuleiros, notas de turmas, imagens, jogos e dados organizados em linhas e colunas.

Saída no terminal

```
[[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

Propriedades das matrizes

Linhas e colunas

Os dados possuem duas posições de acesso.

Mutáveis

Os valores podem ser alterados.

Acesso duplo

Utiliza `matriz[linha][coluna]`.

Podem ser irregulares

As linhas podem possuir tamanhos diferentes, embora isso nem sempre seja recomendado.

Podem armazenar tipos diferentes

Apesar de matrizes numéricas geralmente usarem um único tipo.

Percorridas com laços aninhados

Um laço para as linhas e outro para as colunas.

Percorrendo uma matriz

```
matriz = [  
    [1, 2, 3],  
    [4, 5, 6]  
]
```

```
for linha in matriz:
    for valor in linha:
        print(valor, end=" ")
    print()
```

Saída no terminal

```
1 2 3
4 5 6
```

Alterando um elemento

```
matriz = [
    [1, 2],
    [3, 4]
]

matriz[0][1] = 10

print(matriz)
```

Saída no terminal

```
[[1, 10], [3, 4]]
```

Dica: no acesso `matriz[linha][coluna]`, o primeiro índice representa a linha e o segundo representa a coluna.

Comparação geral

Estrutura	Sintaxe	Ordenada?	Mutável?	Permite duplicatas?	Uso principal
Tupla	()	Sim	Não	Sim	Dados fixos
Dicionário	{"chave": valor}	Preserva inserção	Sim	Chaves não; valores sim	Dados identificados por chaves
Conjunto	{valor1, valor2}	Não para acesso por posição	Sim	Não	Eliminar duplicatas e comparar coleções
Matriz	[[...], [...]]	Sim	Sim	Sim	Dados em linhas e colunas

Exercícios propostos

1. Crie uma tupla com os sete dias da semana e mostre o primeiro e o último elemento.
2. Crie um dicionário para armazenar nome, curso e nota de um estudante.
3. Crie dois conjuntos de disciplinas e mostre as disciplinas em comum.
4. Crie uma matriz 3×3 , percorra seus elementos e calcule a soma total.
5. Explique qual estrutura seria mais adequada para armazenar coordenadas geográficas fixas.